

Chapter 2

布林代數與邏輯閘

基本定義

基本定義

- ▶ 凡是一些具有相同特性的物件之聚集，即稱為**集合(set)**，一個集合 S 的**二元運算子(binary operator)**是定義集合 S 中的任何一對元素經此符號運算以後的結果，仍為該集中的一個元素
- ▶ 六個常見的公設 (Six common postulates)
- ▶ 封閉性 (Closure)
- ▶ 結合律 (Associative Law)
- ▶ 交換律 (Commutative Law)
- ▶ 單位元素 (Identity Element)
- ▶ 反元素 (Invers)
- ▶ 分配律 (Distributive Law)

封閉性 (Closure)

- ▶ 若對於集合 S 中任意一對元素，二元運算都能依據其規則產生集合 S 中唯一的一個元素，則稱集合 S 對此二元運算具有封閉性 (closed)。
- ▶ 範例:
- ▶ $N = \{1, 2, 3, 4, \dots\}$
- ▶ $2 + 3 = 5,$
- ▶ $2, 3 \in N, \text{ and } 5 \in N$

結合律 (Associative Law)

- ▶ $(x * y) * z = x * (y * z)$ for all $x, y, z \in S$
- ▶ 符號 $*$ 為二元運算子

交換律 (Commutative Law)

▶ $x * y = y * x$ for all $x, y \in S$

單位元素 (Identity Element)

▶ $e * x = x * e = x$ for every $x \in S$

▶ e 為一個元素

反元素 (Invers)

- ▶ for every $x \in S$, there exists an element $y \in S$ such that $x * y = e$

分配律 (Distributive Law)

- ▶ 若 $*$ 和 $\bullet$ 是集合 S 中的兩個運算子:
$$x * (y \bullet z) = (x * y) \bullet (x * z)$$

布林代數的公理定義

- **公理 (Axiom)**：作為理論基礎，不需要證明，是正確的假設。
- **定理 (Theorem)**：是建立在公理之上，經過嚴謹邏輯推導得出的結論。
- **公理系統 (Axiomatic system)**：一套由公理和所有從這些公理導出的定理組成的完整體系。

布林代數的公理定義

- ▶ 布林代數是一種代數結構，定義在集合 B 的元素。此集合具有 $+$ 和 $\cdot$ 兩種二元運算符號，並滿足下列(Huntington) 假說：

1.

- ▶ (a) 布林代數結構對於運算元 $+$ 具有封閉性。
- ▶ (b) 布林代數結構對於運算元 $\cdot$ 具有封閉性。

2.

- ▶ (a) 對於運算元 $+$ 具有單位元素 0 ：

$$x + 0 = 0 + x = x$$

- ▶ (b) 對於運算元 $\cdot$ 具有單位元素 1 ：

$$x \cdot 1 = 1 \cdot x = x$$

布林代數的公理定義

3.

- ▶ (a) 布林代數結構對於 $+$ 具有交換律：

$$x + y = y + x$$

- ▶ (b) 布林代數結構對於 $\cdot$ 具有交換律：

$$x \cdot y = y \cdot x$$

4.

- ▶ (a) 運算元 $\cdot$ 對 $+$ 具有分配律：

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$$

- ▶ (b) 運算元 $+$ 對 $\cdot$ 具有分配律：

$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

布林代數的公理定義

5.

► 對任一元素 $x \in B$ ，必存在一元素 $x' \in B$ （稱為 x 的補數，complement），使得

► (a)

$$x + x' = 1$$

► (b)

$$x \cdot x' = 0$$

6.

► 至少存在兩個元素 $x, y \in B$ ，使得

$$x \neq y$$

二值布林代數

► $B = \{0,1\}$

► 運算規則:

x	y	$x \cdot y$	x	y	$x+y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

亨廷頓假設

交換律和分配律

x	y	z
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$y + z$	$x \cdot (y + z)$
0	0
1	0
1	0
1	0
0	0
1	1
1	1
1	1

$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	1	1
1	0	1
1	1	1

布林代數的基本定理與性質

布林代數的基本定理與假設

Postulate 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulate 5	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Theorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Theorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Theorem 3, involution		$(x')' = x$		
Postulate 3, commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Theorem 4, associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Postulate 4, distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Theorem 6, absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

對偶原理 (Duality Principle)

對偶原理的意思是：

如果一個布林代數公式是成立的，那麼把公式中的 **AND 與 OR 互換**，**0 與 1 互換**，所得到的新公式仍然成立。

這就是所謂的 **對偶式 (dual expression)**。

Theorem 1(a)

$$x + x = x$$

Statement	Justification
$x + x = (x + x) \cdot 1$	Postulate 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	2(a)



Duality

Theorem 1(b)

$$x \cdot x = x$$

Statement	Justification
$x \cdot x = (x \cdot x) + 0$	Postulate 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

Theorem 2(a)

$$x + 1 = 1$$



Theorem 2(b)

$$x \cdot 1 = 0$$

Statement	Justification
$x + 1 = 1 \cdot (x + 1)$ $= (x + x')(x + 1)$ $= x + x' \cdot 1$ $= x + x'$ $= 1$	Postulate 2(b) 5(a) 4(b) 2(b) 5(a)

Theorem 6(a)

$$x + xy = x$$

Statement	Justification
$x + xy = x \cdot 1 + xy$ $= x(1 + y)$ $= x(y + 1)$ $= x \cdot 1$ $= x$	Postulate 2(b) 4(a) 3(a) Theorem 2(a) 2(b)

By means of truth table

x	y	xy	$x + xy$	$x + y$	$x(x + y)$
0	0	0	0	0	0
0	1	0	0	1	0
1	0	0	1	1	1
1	1	1	1	1	1

Duality

Theorem 6(b)

$$x(x + y) = x$$

DeMorgan's Theorems

► $(x + y)' = x'y'$

► $(xy)' = x' + y'$

x	y	$x + y$	$(x + y)'$	x'	y'	$x'y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

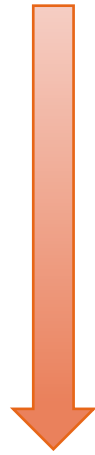
迪摩根定律 (DeMorgan's Theorem)

- ▶ $(A \cdot B)' = A' + B'$
- ▶ $(A + B)' = A' \cdot B'$
- ▶ 三個變數的例子
- ▶ 迪摩根定律也可以延伸：
- ▶ $(ABC)' = A' + B' + C'$
- ▶ $(A + B + C)' = A'B'C'$

運算子優先順序

Highest precedence

Lowest precedence



Parentheses 括號
NOT
AND
OR

Practice Exercise 2.1

Using the basic theorems and postulates of Boolean algebra, simplify the following Boolean expression:

$$F = x'y'z + xyz + x'yz + xy'z$$

Answer:

$$\begin{aligned} F &= x'y'z + xyz + x'yz + xy'z \\ &= (x'y' + xy + x'y + xy')z \\ &= (x'(y' + y) + x(y + y'))z \\ &= (x' + x)z \\ &= z \end{aligned}$$

Practice Exercise 2.2

Develop a truth table for the Boolean expression

$$F = x'y'z$$

Answer:

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

布林函數

► 布林函數的組成

1. 二元變數
2. 二元運算子OR 與AND
3. 單元運算子NOT
4. 括號

► 範例

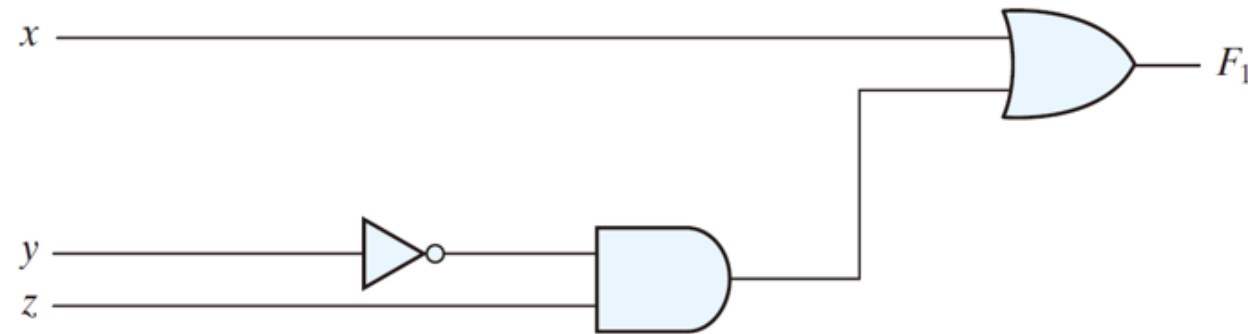
► $F_1 = x + y'z$

► $F_2 = x'y'z + x'y z + xy'$

真值表 Truth Tables

x	y	z	F_1	F_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

邏輯電路

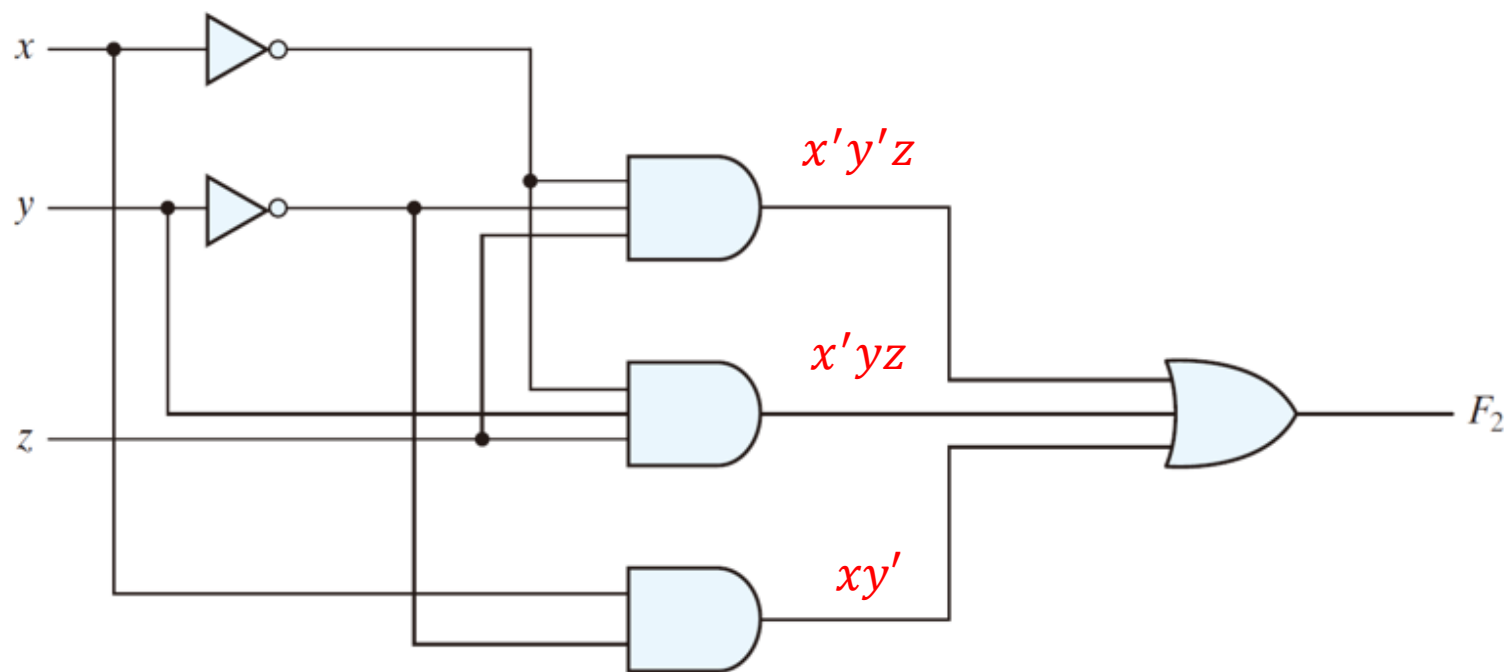


$$F_1 = x + y'z$$

Figure 2.1
Logic diagram for the Boolean function

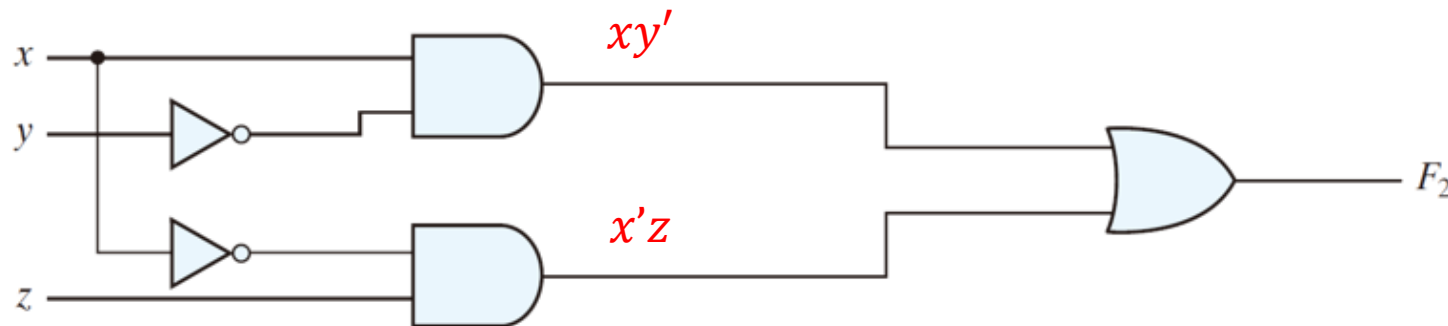
使用邏輯閘電路實現布林函數

$$F_2 = x'y'z + x'yz + xy'$$



使用邏輯閘電路實現布林函數

$$\begin{aligned} F_2 &= x'y'z + x'yz + xy' \\ &= x'z(y' + y) + xy' \\ &= x'z + xy' \end{aligned}$$



代數演算

- ▶ 當布林函數利用邏輯閘來實現時，每一個項需要一個閘，而每一項中的每個變數即為閘的輸入。
 - ▶ 定義一個**文字字元(literal)** 做為每一項中的一個變數，可以是補數，也可以不是。
- ▶ 當我們用邏輯閘（**logic gates**）來實現一個布林函數時，布林表示式中的每一項（**term**）通常對應一個邏輯閘。
- ▶ 例如：

$$F = x'y'z + xyz$$

- ▶ 這裡有兩個項，因此可能需要兩個 **AND** 閘，再接到一個 **OR** 閘。而每一項中的變數，就對應到邏輯閘的輸入。

布林函數化簡範例

1. $x(x' + y) = xx' + xy = 0 + xy = xy$
2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y$
3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x$
4.
$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z \end{aligned}$$
5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$

表示式1 和2 互為對偶，在相對應的每個步驟利用對偶式即可求得
表示式4 及表示式5 就是著名的**重合定理(consensus theorem**；一致定理)。

Consensus Theorem (重合定理)

- ▶ 當一個算式中的某兩個運算元的一部分，各為後續運算元的一部分時，該運算元可以忽視，此即 Consensus Theorem

$$xy + x'z + yz = xy + x'z$$

- ▶ 在這個等式中， yz 就是所謂的「重合項」(consensus term)。

- ▶ • 如何辨認重合項？

觀察等式左邊，有兩項分別包含了某個變數及其反相 (x 與 x')。這兩項剩下的變數相乘 (即 $y \cdot z$)，就會形成第三項。這個第三項就是可以被省略的重合項。

- ▶ 代數證明：

- ▶ 我們可以透過引入 $(x + x') = 1$ 來證明 yz 是多餘的：

$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz = xy(1 + z) + x'z(1 + y) \\ &= xy(1) + x'z(1) = xy + x'z \end{aligned}$$

一個函數的補數

- ▶ 函數 F 的補數為 F' ，可以由 F 值中之0 換成1，1 換成0 求得。
- ▶ 可以利用迪摩根定理：

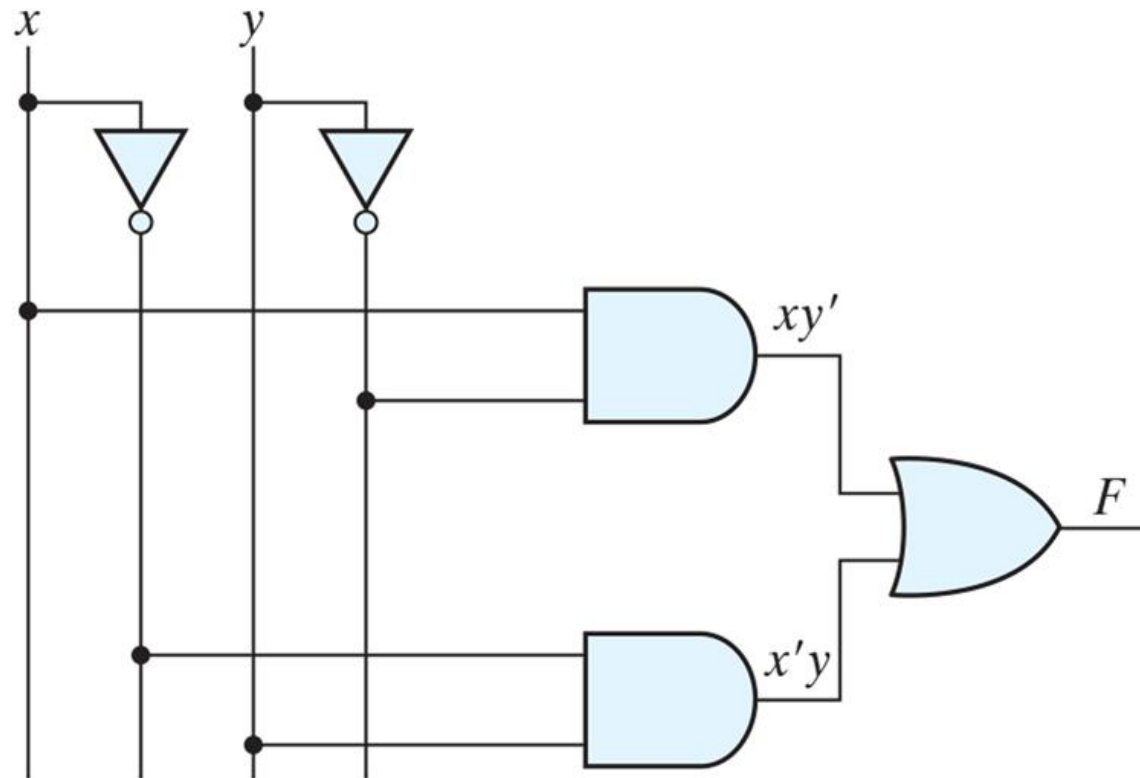
$$\begin{aligned}(A + B + C)' &= (A + X)' && \text{Let } B + C = X \\ &= A'X' && \text{By DeMorgan's} \\ &= A'(B + C)' \\ &= A'(B'C') && \text{By DeMorgan's} \\ &= A'B'C' && \text{associative}\end{aligned}$$

- ▶ 迪摩根定理通式

- ▶ $(A + B + C + \dots F)' = A'B'C' \dots F'$
- ▶ $(ABC \dots F)' = A' + B' + C' \dots + F'$

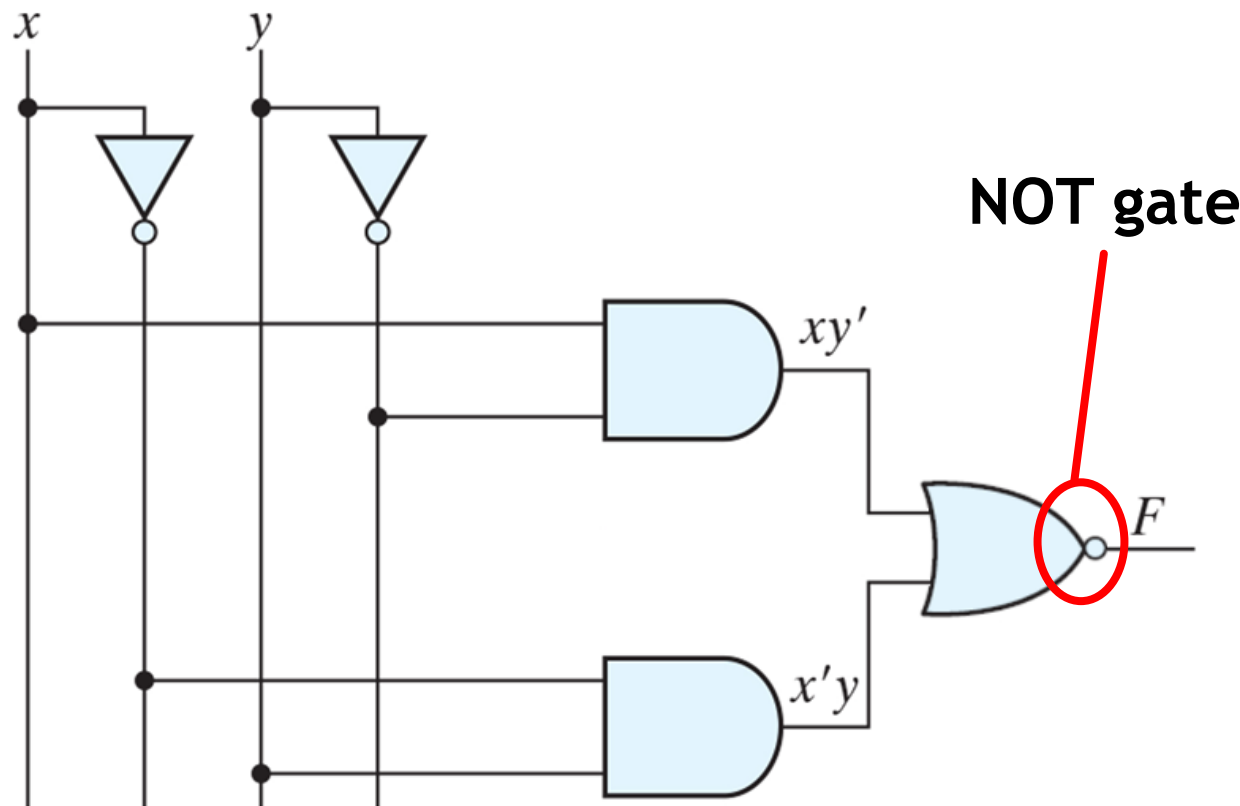
Practice Exercise 2.3

畫初布林函數 $F = x'y + xy'$ 之邏輯電路圖



Practice Exercise 2.4

下列邏輯電路所實現的布林表示式為何？



$$\begin{aligned} F &= (x'y + xy')' \\ &= (x'y)'(xy')' \\ &= (x + y')(x' + y) \\ &= xx' + xy + x'y' + yy' \\ &= xy + x'y' \end{aligned}$$

正規形式與標準形式

- ▶ 全及項(minterm ; 最小項) 、全或項(maxterms ; 最大項) 。
- ▶ minterm 是所有變數的 AND 組合 ; maxterm 是所有變數的 OR 組合 。
- ▶ 正規形式與標準形式:
- ▶ 在布林代數中，一個布林函數可以用一些固定結構的項 (terms) 來表示，例如：
 - ▶ AND 組合
 - ▶ OR 組合
- ▶ 當每個變數都完整出現在一項裡時，就稱為 正規形式 (canonical form) 。

Table 2.3*Minterms and Maxterms for Three Binary Variables*

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

全及項與全或項(最小項與最大項)

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

任何布林函數可以用**全及項的和**來表示(所謂的「和」，即指將各項**OR** 起來)

<i>x</i>	<i>y</i>	<i>z</i>		<i>F</i> ₁	<i>F</i> ₂
0	0	0	<i>m</i> ₀	0	0
0	0	1	<i>m</i> ₁	1	0
0	1	0	<i>m</i> ₂	0	0
0	1	1	<i>m</i> ₃	0	1
1	0	0	<i>m</i> ₄	1	0
1	0	1	<i>m</i> ₅	0	1
1	1	0	<i>m</i> ₆	0	1
1	1	1	<i>m</i> ₇	1	1

The complement of a Boolean function

$$\begin{aligned}f_1' &= m_0 + m_2 + m_3 + m_5 + m_6 \\&= x'y'z' + x'yz' + x'yz + xy'z + xyz'\end{aligned}$$

$$\begin{aligned}f_1 &= (f_1')' \\&= (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z) \\&= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6\end{aligned}$$

任何布林函數可以用全或項的乘積來表示(所謂「乘積」，即為將各項AND 起來)

Example 2.4

Express the Boolean function $F = A + B'C$ as a sum of minterms

$$\begin{aligned} F &= A + B'C \\ &= A(B + B') + (A + A')B'C \\ &= AB + AB' + AB'C + A'B'C \\ &= AB(C + C') + AB'(C + C') + AB'C + A'B'C \\ &= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C \\ &= A'B'C + AB'C' + AB'C + ABC' + ABC \\ &= m_1 + m_4 + m_5 + m_6 + m_7 \end{aligned}$$

為了方便表示布林函數可用下列簡短的標記法：

$$F(A, B, C) = \sum(1, 4, 5, 6, 7)$$

<i>A</i>	<i>B</i>	<i>C</i>		<i>F</i>
0	0	0	m_0	0
0	0	1	m_1	1
0	1	0	m_2	0
0	1	1	m_3	0
1	0	0	m_4	1
1	0	1	m_5	1
1	1	0	m_6	1
1	1	1	m_7	1

Example 2.5

Express the Boolean function $F = xy + x'z$ as a product of maxterms

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(x' + y)(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

$$\begin{aligned} x' + y &= x' + y + zz' \\ &= (x' + y + z)(x' + y + z') \end{aligned}$$

$$\begin{aligned} F &= (x' + y + z)(x' + y + z')(x + y + z)(x + y' + z)(x + y + z)(x' + y + z) \\ &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\ &= M_0 M_2 M_4 M_5 \end{aligned}$$

$$F(x, y, z) = \prod(0, 2, 4, 5)$$

正規形式的轉換

欲將一種標準形式轉換成另一種標準形式，則將 Σ 與 Π 互換，並將括號內之數字轉換成原式中所缺少之數字即可。

$$F = xy + x'z$$

$$F(x, y, z) = \Sigma(1, 3, 6, 7)$$

$$F(x, y, z) = \Pi(0, 2, 4, 6)$$

Table 2.6

Truth Table for $F = xy + x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Minterms

Maxterms

標準形式

- ▶ 標準形式(standard form)可分為兩種形式：積項和與和項積。

- ▶ 積項和(sum of products)

$$F_1 = y' + xy + x'yz'$$

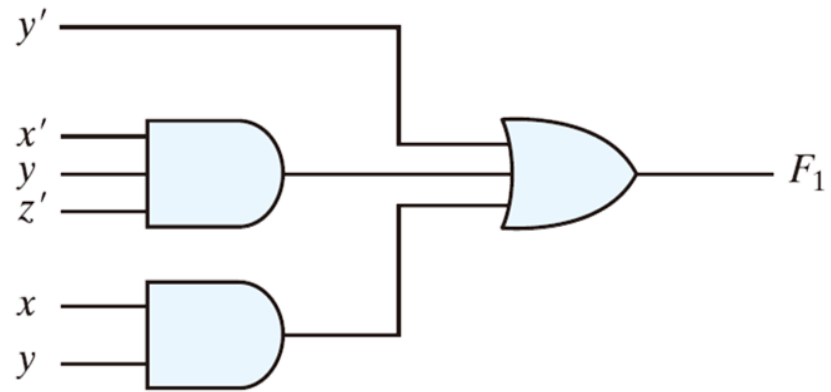
- ▶ 和項積(product of sums)

$$F_2 = x(y' + z)(x' + y + z')$$

$$F_3 = AB + C(D + E) = AB + C(D + E) = AB + CD + CE$$

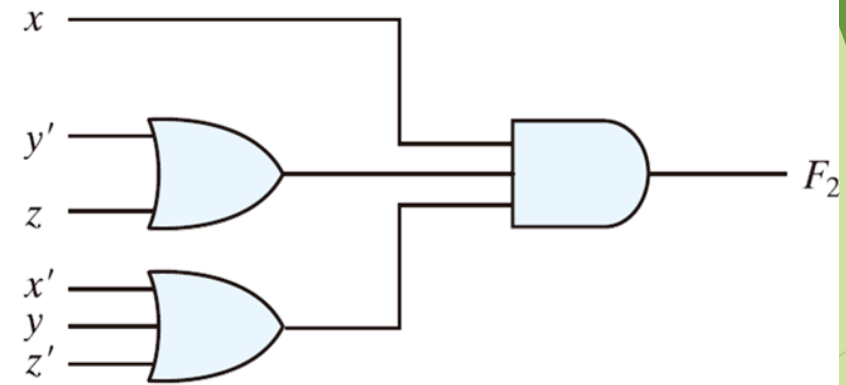
二階層電路

$$F_1 = y' + \textcolor{red}{xy} + x'yz'$$



(a) 積項和

$$F_2 = x(y' + z)(x' + y + z')$$

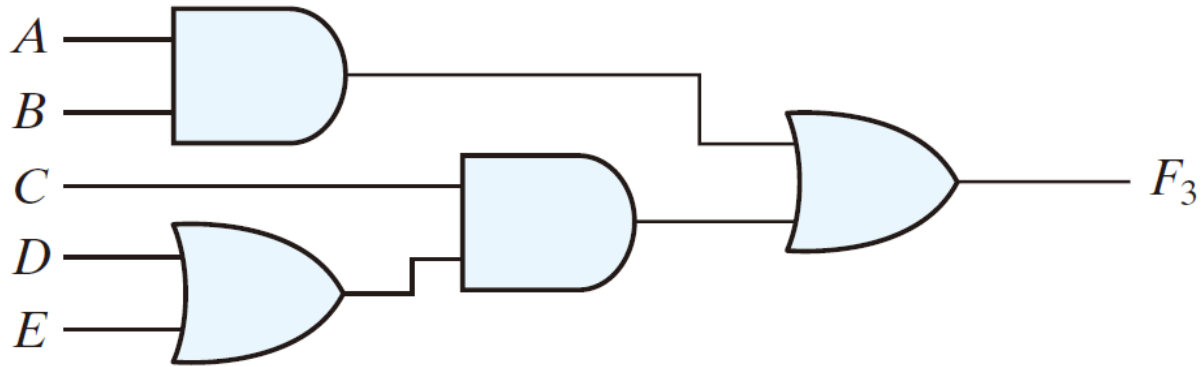


(b) 和項積

圖 2.3 二階層電路

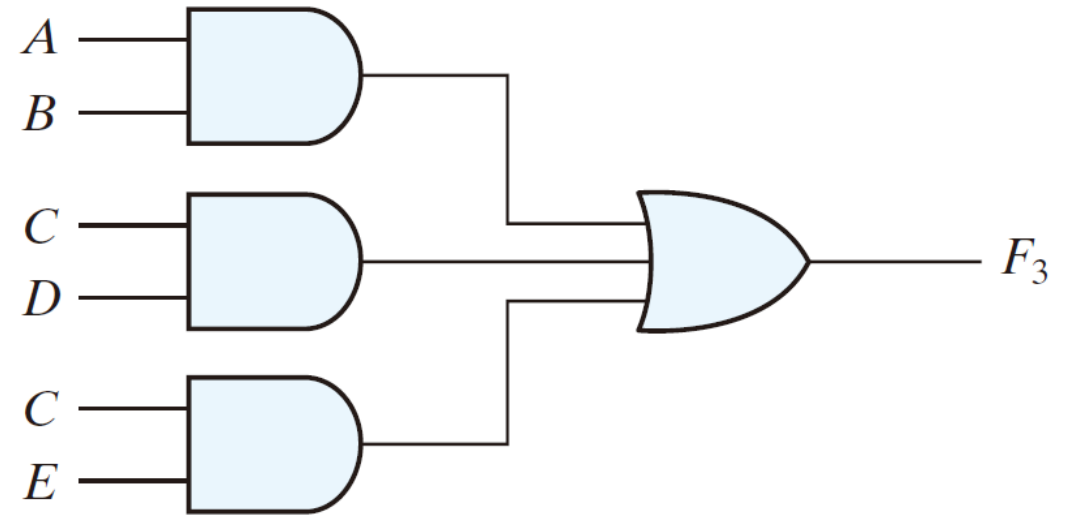
多層電路

$$AB + C(D + E)$$



(a) $AB + C(D + E)$

$$AB + CD + CE$$



(b) $AB + CD + CE$

FIGURE 2.4

Three- and two-level implementation

其他邏輯運算

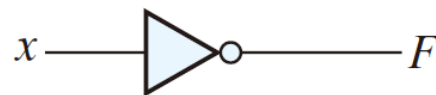
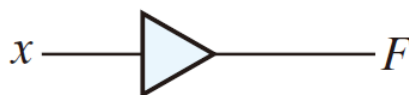
在表2.8 中所列的每個函數在以某種方式解釋時，都會賦予一個名稱以及註解，這**16**個函數可分為三類：

1. 兩個可產生常數**0**或**1**的函數。
2. 單元運算對於**x**或**y**各有補數與轉移兩種，一共四種。
3. 具有二元運算子的函數，一共有八個，包括**AND**、**OR**、**NAND**、**NOR**、互斥-**OR**、全等(equivalence)、禁止(inhibition) 及意含(implication) 等八種運算。

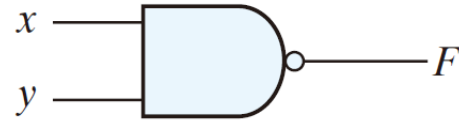
Table 2.8
Boolean Expressions for the 16 Functions of Two Variables

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

數位邏輯閘

Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	

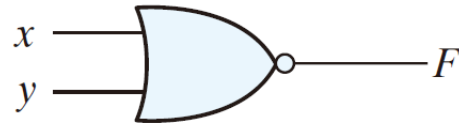
NAND



$$F = (xy)'$$

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

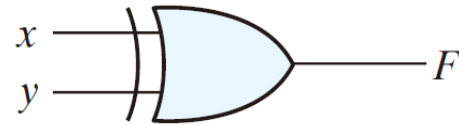
NOR



$$F = (x + y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

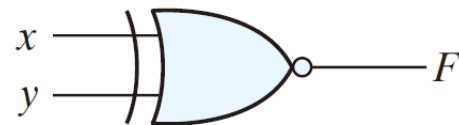
Exclusive-OR
(XOR)



$$F = xy' + x'y$$
$$= x \oplus y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR
or
equivalence



$$F = xy + x'y'$$
$$= (x \oplus y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

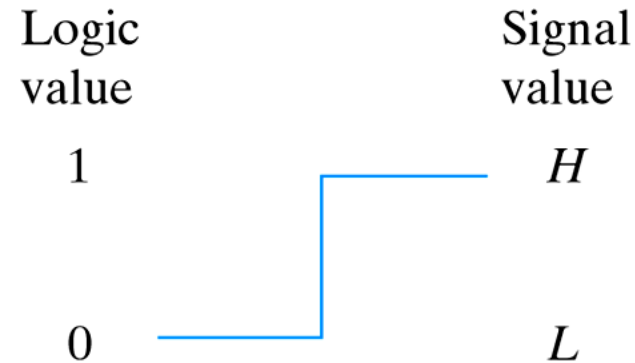
正邏輯與負邏輯

► 正邏輯與負邏輯

- 兩個信號 $\Leftrightarrow$ 兩種邏輯值
- 正邏輯： $H=1$ 、 $L=0$
- 負邏輯： $H=0$ 、 $L=1$



(b) Negative logic



(a) Positive logic

FIGURE 2.9
Signal assignment and logic polarity

x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

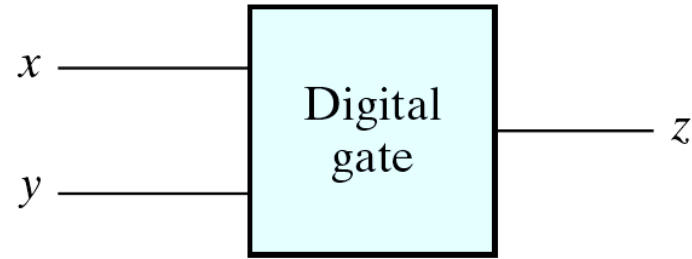
(a) Truth table with H and L

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

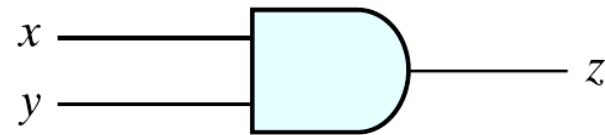
(c) Truth table for positive logic

x	y	z
1	1	1
1	0	1
0	1	1
0	0	0

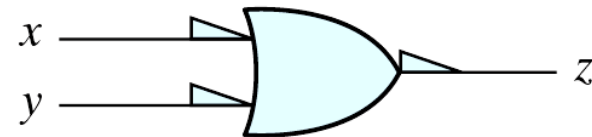
(e) Truth table for negative logic



(b) Gate block diagram



(d) Positive logic AND gate



(f) Negative logic OR gate

在輸入及輸出端的小三角形代表極性指示
(polarity indicator)

FIGURE 2.10

Demonstration of positive and negative logic

積體電路

- ▶ An IC (a chip)
- ▶ Examples:
 - ▶ SSI: < 10 gates
 - ▶ MSI: 10 ~ 100 gates
 - ▶ LSI: 100 ~ xk gates
- ▶ VLSI: > xk gates
 - ▶ small size (compact size)
 - ▶ low cost
 - ▶ low power consumption
 - ▶ high reliability
 - ▶ high speed

數位邏輯家族

1. TTL：電晶體-電晶體邏輯 (transistor-transistor logic)
2. ECL：射極-耦合邏輯 (emitter-coupled logic) (高速、高功率消耗)
3. MOS：金屬-氧化物半導體 (metal-oxide semiconductor) (高密度)
4. CMOS：互補式金屬-氧化物半導體 (complementary metal-oxide semiconductor) (低功率)